The concepts behind the tasks

What a heap is, how malloc works, what exit does, what a register is for …

Read the section before you start a task

The guide (`guide.pdf`) tells you *where* to change SWEB. This document explains *what* the things are that the tasks are about, in plain words, with your team's code as the example. Every section has the same parts: the idea in one sentence, how it works, where it is in your team's code, typical mistakes, three questions to check yourself (answers below them), and the tasks that need it. Deeper background: the foundations handbook (`1-foundations/theory/handbook.pdf`).

A self-study companion for Operating Systems at TU Graz, winter semester 2026/27

Contents

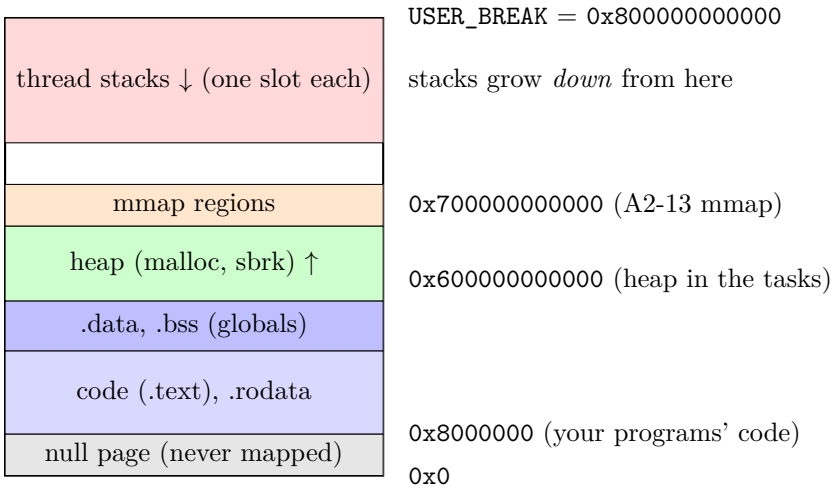
The concepts **2**

1 The address space of a process . . . 2
2 The heap and how malloc works . . . 3
3 Program break, sbrk and demand paging . . . 3
4 Page faults as a tool . . . 4
5 Ending a program: exit, _exit, atexit . . . 5
6 Processes: fork, exec, zombies, waitpid . . . 5
7 C strings . . . 6
8 Registers, the calling convention and the stack frame . . . 7
9 Diverting a thread: changing its saved registers . . . 7
10 Interrupts and the interrupt entry . . . 8
11 Time: timer ticks, time slices, the scheduler . . . 9
12 Atomic instructions, spinlocks and mutexes . . . 9
13 Semaphores, condition variables and the lost wake-up . . . 10
14 Barriers . . . 10
15 A thread's life in your team's code . . . 11
16 Physical pages, reverse lookups and shared pages . . . 11

The concepts

1 The address space of a process

In one sentence: every process sees its own private range of addresses from 0 up to `USER_BREAK`, and different regions of it hold different things.



How it works. The address space is only a promise: an address is usable only if the page tables map its page to a physical frame. Most of the space is not mapped at all. The *code*, *read-only data* and *initialised data* come from the program file (the ELF binary); *.bss* (globals that start at 0) takes no space in the file. The *heap* is memory the program asks for while it runs; the *stacks* hold the local variables and return addresses of every thread. Above `USER_BREAK` is the kernel — the same for every process, not accessible from user mode.

In your team's code. `UserProcess::getAddressSpace()` is the `ArchMemory` (the page tables) of the process. Pages of the binary are mapped lazily by `Loader::loadPage` on the first access; every thread gets a stack slot of 4 pages + 1 guard page, counted down from `USER_BREAK`.

Typical mistakes. Thinking "virtual address = physical address". Forgetting that an unmapped address page-faults even though it is "in the heap region".

Check yourself

1. Why does a global array of 1 MiB not make the program file 1 MiB bigger?
2. Two processes both use address `0x600000000000`. Do they share memory?
3. Where does a thread's local variable live, where a `malloc`ed block?

Answers: 1. it is in .bss: only its size is in the file, the pages are zero-filled on first use. 2. no – each process has its own page tables; the same virtual address maps to different frames. 3. on that thread's stack; on the heap.

Tasks: A0-03, A1-12, A1-14, A2-13 (and every task that touches pages)

2 The heap and how malloc works

In one sentence: the heap is a big area of memory from which `malloc` cuts out blocks of any size on request, and `free` gives them back so they can be reused.

Why it exists. Locals live only until their function returns (stack), globals have a fixed size (data/bss). A program that does not know beforehand how much memory it needs — a list that grows, a string read from input — asks for it at run time: `p = malloc(100)`.

How an allocator works. The heap is one long piece of memory. `malloc` has to remember which parts are in use. The classic trick: a small **header** right before every block, holding its size and whether it is used:



- `malloc(n)`: walk the blocks; the first free block that is big enough is marked used (*first fit*). None? Append a new block at the end. Return the address *after* the header.
- `free(p)`: go back to the header (`(header*) p - 1`) and mark it free. Nothing is given back to the system — the block is just reusable.
- `calloc`: `malloc` + fill with zeros (a reused block contains old data!). `realloc`: if the block is too small, `malloc` a new one, copy, `free` the old one.

Fragmentation. After many `malloc`/`free` the heap is a patchwork: lots of free memory in total, but split into pieces too small for a big request. Remedies: merge neighbouring free blocks (*coalescing*), split big free blocks.

Where the heap memory comes from. Either a fixed area that already exists (a static array — A1-14), or the kernel moves the end of the heap, the *program break*, further up on request (`sbrk`, next section).

In your team's code. The libc `malloc`, `free`, `calloc`, `realloc` in `userspace/libc/src/stdlib.c` are stubs (return 0) — A1-14 fills them in, entirely in user space. The kernel has its own heap for `new`/`delete` (`KernelMemoryManager`) — a different thing in a different address space.

Typical mistakes. Returning the header's address instead of the address after it. Forgetting alignment (blocks should start at multiples of 16). `calloc` without zeroing. Using a block after `free` (use after free), freeing twice (double free).

Check yourself

1. How does `free(p)` know how big the block is?
2. Why does `calloc` have to zero the memory even though fresh heap pages are zero?
3. What is fragmentation, and how can it make `malloc` fail although enough memory is free?

Answers: 1. from the header right before `p`. 2. a reused block contains what the previous owner wrote. 3. free memory split into pieces that are each too small for the request.

Tasks: A1-14 (malloc); background for A0-03, A1-12

3 Program break, sbrk and demand paging

In one sentence: the program break is the current end of the heap; `sbrk(n)` moves it by n bytes, and the kernel only provides the memory when it is actually touched.

How it works. The kernel stores one number per process: the break. `sbrk(+4096)` moves it up and returns the *old* end — the start of the new memory. Nothing is mapped yet! When the program touches an address below the break for the first time, the CPU raises a page fault (the page is not mapped); the kernel's page fault handler sees "below the break", takes a zeroed frame, maps it, and the instruction runs again. Touching an address *above* the break: the handler finds no reason to map it and kills the process.

This "map on first use" is called **demand paging** (or lazy allocation). It is cheap: a program that reserves 100 MiB but uses 1 MiB only costs 1 MiB.

Shrinking. `sbrk(-n)`: pages that now lie completely above the break are unmapped (their frames freed) and the TLB is flushed — otherwise the CPU could still use the old mapping.

In your team's code. Per-process state belongs into `UserProcess` (`brk_`); the page fault handler's new case goes *first* in the valid branch of `handlePageFault`, before it takes `thread_lock_`. The libc `sbrk` stub is in `unistd.c`. A0-03 (heaplimit) is the same pattern with a fixed size instead of a moving break.

Typical mistakes. Mapping pages in the syscall instead of on demand. Checking "below the limit" and mapping in two separate critical sections (another thread can shrink the heap in between). Forgetting the TLB flush when shrinking.

Check yourself

1. What does `sbrk(0)` return, and what is it good for?
2. Why does growing the heap cost almost nothing?
3. Thread A faults on a heap page while thread B shrinks the heap below it. What must your lock guarantee?

Answers: 1. the current break – the start of the next allocation. 2. nothing is mapped until a page is touched. 3. the check "below the break" and the `mapPage` happen in one critical section, so B's shrink either happens before (A is killed) or after (B unmaps the page).

Tasks: A0-03 heaplimit, A1-12 sbrk; the same idea in A2-13 mmap

4 Page faults as a tool

In one sentence: a page fault is the CPU saying "I cannot use this address right now" — and the kernel decides whether that is an error or an opportunity.

How it works. The CPU translates every address through the page tables. If the page is not present, or the access is not allowed (write to read-only, execute with NX), it stops the instruction and jumps into the kernel with the faulting address (register `cr2`) and an error code (present? write? user mode? instruction fetch?). The handler can:

- **fix it and retry:** load the page from the program file (lazy loading), map a fresh page (stack growth, heap, `sbrk`, `mmap`), read it back from swap, copy a shared page (copy-on-write) — then return; the same instruction runs again and now succeeds;
- **kill the process:** null pointer, kernel address, outside every region.

In your team's code. `pageFaultHandler` (`InterruptUtils.cpp`) → `PageFaultHandler::handlePageFault`: `checkPageFaultIsValid` (null page, kernel address, present page → exit 9999); then stack growth (`isUserStackFault`, under `thread_lock_`), guard page (kill), else `Loader::loadPage` (no segment → exit 666).

Typical mistakes. Returning from the handler without fixing the cause: the instruction faults again, forever. Taking a lock in the handler that the faulting code already holds (deadlock). Exiting while holding a lock.

Check yourself

1. Why does the first instruction of every program page-fault?
2. Exit code 666 vs. 9999 in SWEB?
3. A syscall writes into a user buffer that is not mapped yet. Who handles that fault?

Answers: 1. nothing is loaded before – the code page is loaded on demand. 2. 666: no ELF segment covers the address; 9999: invalid fault (null, kernel address, protection). 3. the same page fault handler – the fault comes from kernel mode, but the address is a user address.

Tasks: A0-03, A1-12, A1-11, A1-10, every A2 task

5 Ending a program: exit, _exit, atexit

In one sentence: `exit(code)` first runs the clean-up work the program registered (`atexit`), then asks the kernel to end the process; `_exit(code)` skips the clean-up.

The two layers.

- **libc layer (user space):** `exit` runs the functions registered with `atexit`, in *reverse* order of registration (like a stack: what was set up last is cleaned up first), and would flush buffered output. Then it makes the exit syscall.
- **kernel layer:** the exit syscall ends every thread of the process, frees its memory, remembers the exit code for the parent (`waitpid`).

Returning from `main` is the same as calling `exit`: the start code calls `exit(main())`.

atexit is pure libc. The kernel knows nothing about it: an array of function pointers and a counter in the libc, filled by `atexit`, run by `exit`. A handler that itself calls `exit` must not run the same handlers again — take each one off the list before calling it.

In your team's code. `_start` in `nonstd.c`: `exit(main())`; `exit` and `_exit` in `exec.c` (both only make the syscall today); `atexit` stub in `stdlib.c`. Kernel: `Syscall::exit` → `UserProcess::exitProcess` (kills all threads) → the last thread's destructor deletes the process.

Typical mistakes. Putting `atexit` into the kernel (there is no need). Calling the handlers in registration order. Running them in `_exit`.

Check yourself

1. Why reverse order?
2. What happens to the handlers if the process is killed by a page fault?
3. Why is no syscall needed for `atexit`?

Answers: 1. later registrations may depend on earlier ones – clean up like a stack. 2. they do not run – the kernel ends the process directly, `exit()` in the libc is never reached. 3. it only stores function pointers in the process' own memory and calls them before the exit syscall.

Tasks: A0-13 atexit; background for A1-19 waitpid

6 Processes: fork, exec, zombies, waitpid

In one sentence: `fork` makes a copy of the calling process, `exec` replaces the program inside a process, and `waitpid` lets a parent wait for a child and collect its exit code.

fork. The child gets a copy of the address space (all pages, or shared pages with copy-on-write), the file descriptors, and *one* thread: a copy of the caller. Both continue after the `fork` call — the child sees return value 0, the parent the child's pid.

exec. Same process (same pid), new program: the old address space is thrown away, the new binary is loaded, the calling thread restarts at the new entry point. Other threads end. If loading fails, exec returns an error and the old program continues — so load first, destroy later.

Zombies and waitpid. When a child ends, its memory goes — but its exit code must survive until the parent asks for it. A process that has ended but has not been waited for is a *zombie*: only its record (pid, exit code) is left. `waitpid(pid, &status, 0)` waits until the child has ended, takes the code, and removes the record. A child whose parent ends first is an *orphan*; nobody can wait for it any more.

In your team's code. `UserProcess::fork` (deep copy via the `ArchMemory` copy constructor, child thread with `rax = 0`), `UserProcess::exec`; pids from `ProcessRegistry`. `waitpid` is a stub in `wait.c` (Felix is working on it); the course task A1-19 keeps the exit codes in a fixed table in the `ProcessRegistry`, filled in `~UserProcess`.

Typical mistakes. Keeping the exit code in the process object (it is deleted). Copying all threads in fork. Destroying the old address space in exec before the new program is loaded.

Check yourself

1. Why does the child of `fork` see 0 and the parent the pid?
2. Why can't the exit code simply stay in the `UserProcess`?
3. What is the difference between a zombie and an orphan?

Answers: 1. the kernel puts different return values (`rax`) into the two copies of the thread. 2. the process object is deleted when its last thread is gone – long before the parent may ask. 3. zombie: ended, parent has not waited yet; orphan: its parent has ended.

Tasks: A1-19 waitpid; background for fork/exec in your team

7 C strings

In one sentence: a C string is just bytes in memory, ending with a byte 0 (the *NUL*) — nothing stores its length.

How it works. `"abc"` is 4 bytes: `'a' 'b' 'c' 0`. Every string function walks until it finds the 0. Consequences:

- `strlen` counts up to the 0; `strcpy(d, s)` copies including the 0 — if `d` is too small, it writes past its end (buffer overflow);
- `strncpy(d, s, n)` writes *exactly* n bytes: the string, then zeros — but if `s` has n or more characters, *no* terminating 0 is written;
- `strncmp` compares at most n characters and stops at a 0; compare as `unsigned char` (a plain `char` may be negative);
- `strchr(s, c)` returns a pointer to the first `c` (also finds the 0 for `c = 0`).

Strings and the kernel. A user string passed to a syscall has an unknown length: the kernel must copy it byte by byte and check every address against `USER_BREAK` (`Syscall::copyUserString`) — a plain `strcpy` could read kernel memory.

In your team's code. `userspace/libc/src/string.c` has `memcmp`, `memcpy`, `memset`, `strlen`, `strcmp`, `strstr` — `strcpy`, `strncpy`, `strcat`, `strncmp`, `strchr` are missing, not even declared in `string.h`.

Check yourself

1. How many bytes does `"hello"` take?
2. `strncpy(buf, "hello", 3)` — what is in `buf`?
3. Why must the kernel not `strcpy` a string from user space?

Answers: 1. 6. 2. `'h' 'e' 'l'` – no terminating 0. 3. it does not know where the string ends: an unterminated string near `USER_BREAK` would make it read kernel memory.

Tasks: A0-14 string functions; A0-11 (a user string in a syscall)

8 Registers, the calling convention and the stack frame

In one sentence: functions pass their first arguments in registers, the return address lives on the stack, and every thread has its own set of registers and its own stack.

The registers that matter (x86-64).

`rip`	instruction pointer: the address of the next instruction
`rsp`	stack pointer: the top of the stack (the stack grows down)
`rdi rsi rdx rcx r8 r9`	arguments 1–6 of a C function call; arguments 7+ go on the stack
`rax`	return value
`rbp`	frame pointer (start of the current function's stack frame)
`cr3`	physical address of the page tables of the running process

A call. `call f` pushes the return address (the instruction after the call) onto the stack and jumps to `f`; `ret` pops it and jumps back. Below `rsp`, a function may use 128 bytes without moving `rsp` (the *red zone*) — so if the kernel pushes something onto a user stack, it skips those 128 bytes first. At a call, `rsp` is 16-byte aligned.

Syscalls are different. SWEB's `__syscall` passes the number in `rax` and the arguments in `rbx rcx rdx rsi rdi` — five values. More arguments (A1-13: fifteen) must go into user memory, with their address in one register.

In your team's code. When a thread enters the kernel, all its user registers are saved into `currentThread->user_registers_` (`ArchThreadRegisters`); when it returns, they are loaded again. A syscall's return value is written into the saved `rax`.

Check yourself

1. Where is the return address of a function?
2. A function has 8 parameters. Where are parameters 7 and 8?
3. Why does a fresh thread stack need a "trampoline" function?

Answers: 1. on the stack, pushed by `call`. 2. on the stack, above the return address. 3. a fresh stack has no return address – if the thread's function returned, it would jump nowhere; the trampoline calls it and then `pthread_exit`.

Tasks: A1-13 sum15, A0-07 upcall, A1-08 snapshot, A0-09 pthread_multi

9 Diverting a thread: changing its saved registers

In one sentence: while a thread is in the kernel, its user registers are just a struct in memory — change `rip`, and the thread continues somewhere else when it returns.

How it works. Back to user mode means: load the saved registers. So the kernel can

- **call a user function** (upcall, signal handler, A0-06/A0-07): "push" the old `rip` onto the user stack as a return address (`rsp -= 8`, write it), set `rip = function`, `rdi = argument`. The function runs, and its `ret` jumps back to where the thread was;
- **save a thread's state** (snapshot, A1-08): copy the saved registers *and* the used part of its stack;
- **restore it** (revive): copy both back; the old syscall "returns again" — with a different `rax`.

Writing to the user stack from the kernel. Check that the page is mapped (`checkAddressValid`), skip the red zone, keep 16-byte alignment.

In your team's code. `currentThread->user_registers_->rip / rsp / rdi / rax`; a thread's stack top: `((UserThread*) currentThread)->getUserStackTop()`.

Check yourself

1. Why is the old `rip` pushed onto the user stack?
2. Why must a snapshot also copy the stack, not only the registers?
3. What decides the return value a user program sees from a syscall?

Answers: 1. so the called function's `ret` returns to where the thread was interrupted. 2. the saved `rsp` points into the stack – frames written since then would be wrong. 3. the saved `rax` when the thread returns to user mode.

Tasks: A0-07 upcall, A1-08 snapshot, A0-06 segv handler

10 Interrupts and the interrupt entry

In one sentence: an interrupt makes the CPU stop what it does, save a little state and jump to a handler the kernel registered — for a timer tick, a key, a page fault or a syscall.

How it works. The *IDT* (interrupt descriptor table) has one entry per vector (0–255): 14 = page fault, 32 = timer, 33 = keyboard, `0x80` = SWEB's syscall. The CPU switches to the kernel stack, pushes `rip`, `rsp`, flags (and for some exceptions an error code) and jumps to the entry. SWEB's assembly stub saves all other registers, calls the C handler, and `iretq` goes back. During the handler interrupts are *off* (until the handler enables them) — so no Mutex there, only atomic operations or flags.

For a page fault, the CPU writes the faulting address into register `cr2`; the stub passes it to the C handler.

In your team's code. `arch/x86/64/source/arch_interrupts.S` (stubs), `InterruptUtils.cpp` (`syscallHandler`, `pageFaultHandler`, `irqHandler_0` timer, `irqHandler_1` keyboard, `errorHandler`), `InterruptUtils::initialise` fills the IDT.

Check yourself

1. Why can't you take a Mutex in the timer interrupt handler?
2. Where does the page fault handler get the faulting address from?
3. What is the difference between an exception and a hardware interrupt?

Answers: 1. interrupts are off – if the Mutex is taken, the handler would have to sleep, and nothing could ever wake it. 2. register `cr2`, passed on by the assembly stub. 3. an exception is caused by the running instruction (page fault, division by 0); a hardware interrupt comes from a device (timer, keyboard) at any time.

Tasks: A1-11 page fault statistics; background for A1-07, A1-18

11 Time: timer ticks, time slices, the scheduler

In one sentence: a hardware timer interrupts the CPU at a fixed rate; each interrupt is a "tick", and on every tick the scheduler may switch to another thread.

How it works in SWEB. The timer fires 18.2 times a second (1193180 Hz / 65536), so one tick ≈ 55 ms — that is also the time slice. `irqHandler_0` counts the tick (`Scheduler::incTicks`) and calls `Scheduler::schedule()`, which picks the next thread that can run (round robin: the first schedulable one in the list, then rotate the list). `sched_yield` gives the CPU away early.

- **Measuring time:** count ticks — since boot (`getTicks()`), or per thread (add one to the thread that was running when the tick came).
- **Sleeping:** wait until the tick count reached `start + seconds` × 18.2 — simple: yield in a loop; better: mark the thread sleeping and let the scheduler wake it.
- **Changing the policy:** `schedule()` decides — run a thread twice (A1-04), run a chosen thread next (A1-05). It runs with interrupts off: no locks, no `new`/`delete`, only single-word reads/writes.

Check yourself

1. How long is one tick in SWEB?
2. Why may `schedule()` not take a lock?
3. How can a thread's CPU time be measured with ticks?

Answers: 1. about 55 ms (18.2 per second). 2. it runs in the timer interrupt with interrupts off. 3. on every tick, add one to the counter of the thread that was running.

Tasks: A1-18 sleep, A1-07 thread runtime, A1-04 thread flag, A1-05 pthread_invoke

12 Atomic instructions, spinlocks and mutexes

In one sentence: a lock lets only one thread at a time into a critical section, and it needs one instruction that reads and writes memory in a single, indivisible step.

Why "if free, take it" does not work. Between reading "free" and writing "taken" the timer can switch to another thread that also reads "free" — both enter. The CPU offers atomic instructions that do both in one step: `xchg` (swap), compare-and-swap, atomic add. In C (GCC): `__sync_lock_test_and_set(&lock, 1)` writes 1 and returns the old value.

```
while (__sync_lock_test_and_set(&lock, 1))   // old value 1: somebody else has it
  sched_yield();                             // let the holder run, then try again
/* critical section */
__sync_lock_release(&lock);                  // lock = 0
```

Spinlock vs. mutex. A spinlock waits actively (spins, here: yields in a loop). A mutex puts the waiting thread to sleep and the unlocker wakes it — that needs the kernel (the scheduler). SWEB's kernel has both (`SpinLock`, `Mutex`); in user space, your libc's `pthread_mutex_*` and `pthread_spin_*` are stubs (A1-15).

Deeper: handbook chapter 4.

Check yourself

1. Why does a lock need an atomic instruction?
2. Why `sched_yield` in the loop on a single CPU?
3. A test with a non-atomic lock passes. Is the lock correct?

Answers: 1. reading and setting must be one step, else two threads can both see "free". 2. the holder needs the CPU to finish and unlock – spinning would waste the whole time slice. 3. no: the gap is a few instructions, the timer rarely hits exactly there – correctness is an argument, not a test result.

Tasks: A1-15 spinlock + mutex; basis for A1-16, A1-17

13 Semaphores, condition variables and the lost wake-up

In one sentence: a semaphore is a counter you can wait on; a condition variable lets a thread sleep until another thread tells it "something changed".

Semaphore. `sem_wait`: wait until the counter is > 0, then decrement it. `sem_post`: increment it (and so let a waiter continue). Initialised with 1 it is a lock; with n, it allows n at a time; with 0, it counts events (producer/consumer).

Condition variable. Always used with a mutex:

```
lock(m);
while (!condition)          // always a loop: re-check after waking up
  cond_wait(&c, &m);        // releases m, sleeps, takes m again before returning
/* the condition holds, m is held */
unlock(m);
// the other thread: lock(m); make the condition true; cond_signal(&c); unlock(m);
```

The lost wake-up. If a thread checked the condition, then released the lock, and only then started waiting, a signal in between would be lost — it would sleep forever. That is why `cond_wait` releases the mutex and starts waiting *as one step* (or, in a counter-based implementation, remembers the counter before releasing).

In your team's code. The kernel: `Condition` (`wait`, `signal`, `broadcast`) with a `Mutex`; `thread_exited_` in `joinThread` is exactly this pattern. User space: `sem_*` and `pthread_cond_*` are stubs (A1-16, A1-17).

Deeper: handbook chapter 5.

Check yourself

1. Why is `cond_wait` always called in a `while` loop?
2. Signal vs. broadcast?
3. A semaphore initialised with 0 — what is it good for?

Answers: 1. after waking up the condition may already be false again (another thread was faster), and wake-ups may be spurious. 2. signal wakes one waiter, broadcast all – needed when waiters wait for different things. 3. counting events: the consumer waits until a producer has posted.

Tasks: A1-16 semaphore, A1-17 condition variables, A1-03 barrier; your join

14 Barriers

In one sentence: a barrier makes n threads wait until all n have arrived, then lets all of them continue together.

How it works. A counter of arrived threads; the n-th thread to arrive resets it and wakes everybody (broadcast); the others sleep. The barrier must be reusable for the next round — and that is the trap: a woken thread must not check "counter == 0", because the next round may already be filling up. Instead each round has a number (*generation*); a thread waits until the generation has changed.

```
lock(m);
```

```
my_round = generation;
if (++arrived == n) { arrived = 0; generation++; broadcast(c); serial = 1; }
else while (generation == my_round) wait(c, m);
unlock(m);
```

Check yourself

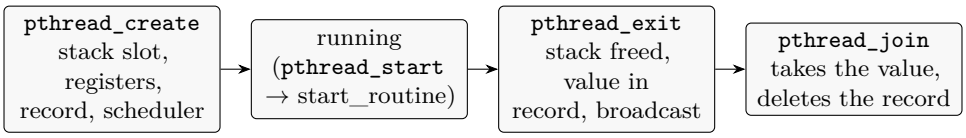
1. Why broadcast, not signal?
2. Why a generation counter instead of checking `arrived`?
3. Exactly one thread gets `PTHREAD_BARRIER_SERIAL_THREAD` — which one in the code above?

Answers: 1. all $n-1$ waiters must wake up. 2. `arrived` may already count the next round when a waiter wakes. 3. the last one to arrive.

Tasks: A1-03 barrier

15 A thread's life in your team's code

In one sentence: a thread is created with its own stack and registers in an existing address space, runs, ends, and leaves a small record behind until somebody has joined it.



- **Shared** by all threads of a process: the address space (code, globals, heap), open files. **Private:** registers, stack, the thread's kernel stack.
- **The record** (`UserThreadData`: tid, exit value, exited, joined, cancel flag) outlives the thread object, which the CleanupThread deletes at some point — join needs the value later. Detach: nobody will join, the record goes when the thread ends.
- **Per-thread data** for a task (A0-11, A1-07, A1-08) goes into `UserThread` (or `Thread`); reach it with `(UserThread*) currentThread`.
- **The lock:** `process->getThreadLock()` protects the thread list, the stack slots and the page tables.

Check yourself

1. What do two threads of the same process share, what not?
2. Why can the exit value not stay in the thread object?
3. Where do you put a field that every user thread needs its own copy of?

Answers: 1. shared: address space, globals, heap, files; private: registers, stack. 2. the CleanupThread deletes it at an unknown time; join may come later. 3. a member of `UserThread`.

Tasks: A1-02, A0-09, A0-11, A1-07, A1-08, A1-20; your join/cancel

16 Physical pages, reverse lookups and shared pages

In one sentence: a virtual page number (vpn) is mapped to a physical frame number (ppn); going from vpn to ppn is one page-table walk, the other direction needs a search — and a frame can be mapped more than once.

vpn → ppn (`vtop`): `resolveMapping(vpn)` walks the four levels of the page table and returns the ppn (or "not mapped"). **ppn → vpn** (`ptov`): page tables only point downwards — walk every present entry of the process and compare.

Shared pages. Two page tables (two processes, or two addresses in one process) can point to the same frame: both see the same bytes. Then freeing becomes the problem: every unmap would free the frame. Solution: a **reference count** per shared frame — unmapping without freeing, decrement the count, free the frame only when it reaches 0. Your team's `~ArchMemory` frees every present frame at exit — a frame mapped twice is freed twice ("Double free PPN").

In your team's code. `ArchMemory::resolveMapping`, `unmapPage(vpn)` (frees the frame), `getIdentAddressOfPPN` (the kernel's view of a frame), `PageManager::allocPPN/freePPN`.

Check yourself

1. Why is ppn → vpn more expensive than vpn → ppn?
2. Two processes share a frame; one exits. What must not happen?
3. What does the ident mapping let the kernel do?

Answers: 1. page tables are a tree from vpn to ppn – the other way needs a search over all entries. 2. its exit must not free the frame while the other process still maps it. 3. read and write any physical frame directly, without mapping it into a process.

Tasks: A1-09 vtop/ptov, A1-10 shared page; background A0-12